Git / GitHub ⭐ ⭐ ⭐ — 3–4 Years Scenario-Based Q&A — Part 2

16. What is git cherry-pick**? When would you use it?**
git cherry-pick is used to apply a specific commit from one branch onto another branch.
For example, a bug fix already exists in the development branch, but Production needs only that particular fix.
Development
↓
Commit A
Commit B ← Bug Fix
Commit C
↓
Cherry-pick B
↓
Production / Hotfix
Instead of merging the entire branch, I can selectively bring only the required commit.
Simple ga gurthupettuko:
Cherry-pick = **One specific commit ni another branch ki teesukovadam**
Eng: I use git cherry-pick when I need to apply a specific existing commit to another branch without merging the complete branch.

---

17. When would you use git rebase**?**
Rebase is mainly used to move my branch on top of the latest target branch and create a cleaner linear history.
Example:
Before:

```
main:    A---B---C
          \
feature:   D---E
```

After Rebase:

```
main:    A---B---C
           \
feature:     D'---E'
```

I normally use rebase on my private feature branch before creating or updating a PR.
I avoid rebasing shared branches because it rewrites commit history.
Simple ga gurthupettuko:
Rebase = **Latest base meeda mana changes replay cheyyadam**
Eng: I use rebase mainly to keep a private feature branch up to date with the target branch and maintain a cleaner history, while avoiding it on shared branches.

---

18. What is the difference between merge and rebase?
Both integrate changes, but they handle history differently.
Merge creates a merge commit when necessary and preserves the existing branch history.
Rebase moves commits onto a new base and rewrites their commit history.
Merge:
A → B → C
↘
D → Merge

Rebase:
A → B → C → D' → E'
Simple ga gurthupettuko:
Merge = **History preserve**
Rebase = **History clean/rewrite**
Eng: Merge preserves the branch history by combining branches, while rebase rewrites commits onto a new base to create a more linear history.

---

19. Your branch is behind main by many commits. What will you do?
First, I make sure my working tree is clean.
Then I fetch the latest changes:
git fetch origin
I can then update my branch using either merge or rebase depending on the team's branching strategy.
For a private feature branch, rebase is often useful:
git rebase origin/main
Then I resolve conflicts if any and run tests before pushing.
Simple ga gurthupettuko:
Behind Main → **Fetch → Update Branch → Resolve → Test**
Eng: I would fetch the latest main branch, update my feature branch according to the team's branching strategy, resolve any conflicts, run tests, and then push the changes.

---

20. What is a detached HEAD state?
Detached HEAD means Git is pointing directly to a commit instead of a branch.
For example:
git checkout <commit-sha>
Now I am viewing that specific commit rather than working on a normal branch.
If I make changes that need to be preserved, I create a branch:
git checkout -b recovery-branch
Simple ga gurthupettuko:
Detached HEAD = **Branch ki attach kakunda specific commit daggara unnappudu**
Eng: A detached HEAD means HEAD points directly to a commit rather than a branch. If I need to preserve new work from that state, I create a new branch.

---

21. You pushed a commit, but now you want to remove it from the remote branch. What will you do?
It depends on whether the branch is shared.
For a shared branch, I prefer:
git revert <commit>
This safely creates a new commit that reverses the change.
If it is my private branch and rewriting history is acceptable, I may use reset followed by a force push, but I use this very carefully.
Simple ga gurthupettuko:
Shared → **Revert**
Private → **Reset may be possible**
Eng: For a shared remote branch, I prefer git revert to avoid rewriting history. I use history-rewriting operations only on private branches and with caution.

---

22. What is git stash **and when do you use it?**
git stash temporarily stores uncommitted changes so I can switch branches or perform another Git operation without committing incomplete work.
Example:
git stash
git checkout main
Later I can restore the changes:
git stash pop
Simple ga gurthupettuko:
Stash = **Temporary ga current changes pakkana petti vere work cheyyadam**
Eng: I use git stash when I have uncommitted changes that I am not ready to commit but need to temporarily set aside.

---

23. A developer says their local branch contains changes that they did not make. What will you check?
First, I check:
git status
git log
git diff
Then I identify whether the changes came from:

- Pull/merge
- Rebase
- Stash
- Another developer
- Generated files
- Local configuration

I avoid deleting anything until I understand where the changes came from.
Simple ga gurthupettuko:
Unknown Changes → **Status → Diff → Log → Understand First**
Eng: I would inspect the working tree, commit history, and diff to identify where the unexpected changes came from before modifying or deleting anything.

---

24. Git push is rejected with "non-fast-forward". What does it mean?
It usually means the remote branch contains commits that my local branch does not have.
For example:
Remote: A → B → C
Local: A → B → D
Git prevents me from overwriting the remote history.
I first fetch the remote changes and integrate them using the team's preferred strategy.
Simple ga gurthupettuko:
Non-fast-forward = **Remote lo mana local lo leni changes unnayi**
Eng: A non-fast-forward rejection means the remote branch has changes that are missing locally, so I first synchronize my branch with the remote before pushing again.

---

25. How do you safely force push?
Force push can overwrite remote history, so I use it very carefully.
If supported by the workflow, I prefer:
git push --force-with-lease

--force-with-lease provides an additional safety check compared with a plain force push.
I would never force push to a protected Production/main branch unless there is an explicitly approved emergency process.
Simple ga gurthupettuko:
Force Push = **Dangerous**
--force-with-lease = **Safer option**
Eng: I use force push only when necessary, preferably --force-with-lease, and I avoid rewriting history on shared or protected branches.

---

26. How do you compare two branches?
I can use:
git diff main..feature
to see code differences.
I can also inspect commits:
git log main..feature
This helps me understand what changes will be introduced before merging.
Simple ga gurthupettuko:
git diff = **Code difference**
git log = **Commit difference**
Eng: I use git diff to compare code changes and git log to compare commit history between branches.

---

27. How do you revert a bad merge commit?
A merge commit has multiple parents, so I need to specify which parent should be treated as the mainline when reverting.
Typically:
git revert -m 1 <merge-commit>
Before doing this in Production, I verify the impact and test the resulting code.
Simple ga gurthupettuko:
Bad Merge → **Identify Merge Commit → Choose Mainline → Revert → Test**
Eng: To revert a merge commit, I identify the correct mainline parent, use git revert -m, and then validate the resulting changes before deployment.

---

28. GitHub Pull Request has conflicts. How will you resolve them?
First, I update my local branch with the latest target branch.
Then I merge or rebase according to team standards.
Latest main
↓
Feature Branch
↓
Resolve Conflict
↓
Run Tests
↓
Push
↓
PR Updated
After resolving conflicts, I verify that the application behavior is still correct.
Simple ga gurthupettuko:
PR Conflict → **Update → Resolve → Test → Push**
Eng: I resolve PR conflicts by bringing the latest target branch into my feature branch, resolving conflicts carefully, running tests, and pushing the updated branch.

---

29. How do you manage Git branches in a DevOps project?
The exact strategy depends on the organization, but a common approach is:
main
│
├── develop
│ ├── feature/*
│ └── bugfix/*
│
└── release/*
└── hotfix/*
For smaller teams, trunk-based development with short-lived feature branches may be preferred.
The important part is that Production deployments are controlled and changes go through review and CI/CD checks.
Simple ga gurthupettuko:
Feature → **PR → CI → Review → Main → Deploy**
Eng: I choose a branching strategy based on team requirements, but I ensure controlled merges, code reviews, automated validation, and a clear path from source control to deployment.

---

30. Production is down after the latest Git change. What is your immediate approach?
I first focus on **restoring service**, not blindly debugging everything.
I check:
Incident

↓
Recent Deployment
↓
Git Commit
↓
Logs + Metrics
↓
Rollback if Required
↓
Service Restored
↓
RCA
If the latest release is confirmed as the cause and rollback is safe, I roll back to the last known-good version.
After recovery, I perform root-cause analysis and implement preventive actions.
Simple ga gurthupettuko:
Production Down → **Restore First → RCA Next**
Eng: During a Production incident, my priority is to minimize impact and restore service safely, usually by validating the recent change and rolling back when appropriate, followed by root-cause analysis.

---

31. How do you ensure developers cannot directly push to Production branch?
I use repository branch protection and access controls.
Typical controls are:

- Pull Request required
- Minimum reviewers
- CI checks required
- Direct push restricted
- Force push restricted
- Required status checks

Developer
↓
Feature Branch
↓
PR
↓
Review
↓
CI
↓
Main
Simple ga gurthupettuko:
Main = **PR + Review + CI**
Eng: I enforce branch protection so Production branches require pull requests, reviews, successful CI checks, and restricted direct or force pushes.

---

32. How do you make Git-based deployments traceable?
I connect every deployment to a specific Git commit.
For example:
Git SHA→ Build Number→ Docker Image Tag→ Deployment→ Production

Example:
app:8f32a1c
Here 8f32a1c can represent the Git commit SHA.
This makes rollback and troubleshooting much easier.
Simple ga gurthupettuko:
Git SHA → Build → Image → Deployment
Eng: I use Git commit-based versioning throughout the CI/CD process so every Production deployment can be traced back to an exact source-code revision.
